Serilog-ui: Interfaz de usuario de visor de registros de Serilog simple para varios receptores.

Arquitectura e Integración de Sistemas Software

Febrero 2026

Javier Martín García (javmargar1@alum.us.es)
Alejandro Márquez Villalba (alemarvil1@alum.us.es)
Gonzalo De la Fuente Hernández (gondeher@alum.us.es)
Pablo García Cantero (pabgarcan1@alum.us.es)
Ángel Nolasco Calderón (angnolcal@alum.us.es)

Tutor: Sergio Rueda Segura
Número de grupo: 3

Link del proyecto en Github: https://github.com/serilog-contrib/serilog-ui

HISTORIAL DE VERSIONES

Fecha	Versión	Descripción	Participantes
01/03/2026	1.0	- Primera versión	Javier Martín García, Alejandro Márquez Villalba, Pablo García Cantero, Gonzalo De La Fuente Hernández, Ángel Nolasco Calderón

Índice

1 Introducción 4

2 Visión general 6

3 Participantes 7

4 Vistas 8

4.1 Vista de contexto 8

4.2 Escenarios de uso 10

4.3 Vista funcional 11

4.4 Vista de despliegue 13

4.5 Vista de Desarrollo 15

5 Puntos de variabilidad y extensión 17

6 Análisis de atributos de calidad 19

7 Sugerencias de mejora 20

8 Contribuciones al proyecto 22

9 Conclusiones 23

Referencias 23

1 Introducción

El proyecto serilog-ui es un complemento para la librería Serilog en el ecosistema .NET. Mientras Serilog genera y almacena registros estructurados (logs) en distintos sistemas o *sinks*, no proporciona una interfaz gráfica para consultarlos. En este contexto, serilog-ui surge como una solución que ofrece una interfaz de usuario accesible desde el navegador, facilitando la visualización, filtrado y análisis de logs almacenados en bases de datos como SQL Server, MySQL, PostgreSQL, MongoDB o SQLite, entre otros. De este modo, permite inspeccionar eventos de la aplicación sin acceder directamente a la base de datos ni utilizar herramientas complejas.

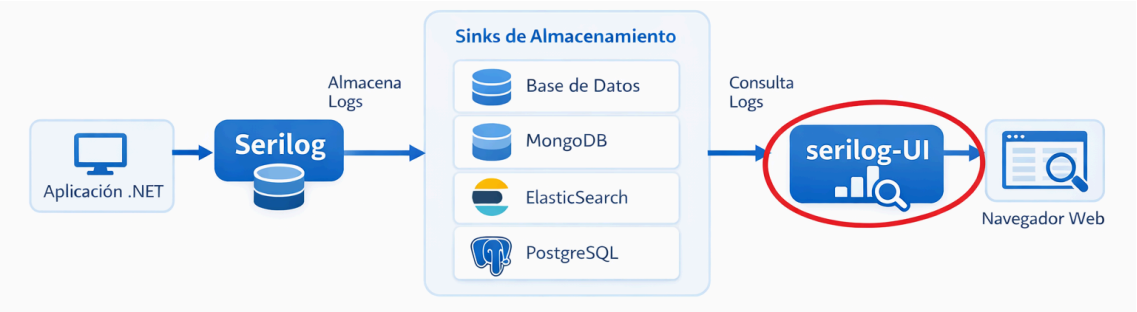

Imagen 1: Flujo del sistema (diagrama generado por IA)

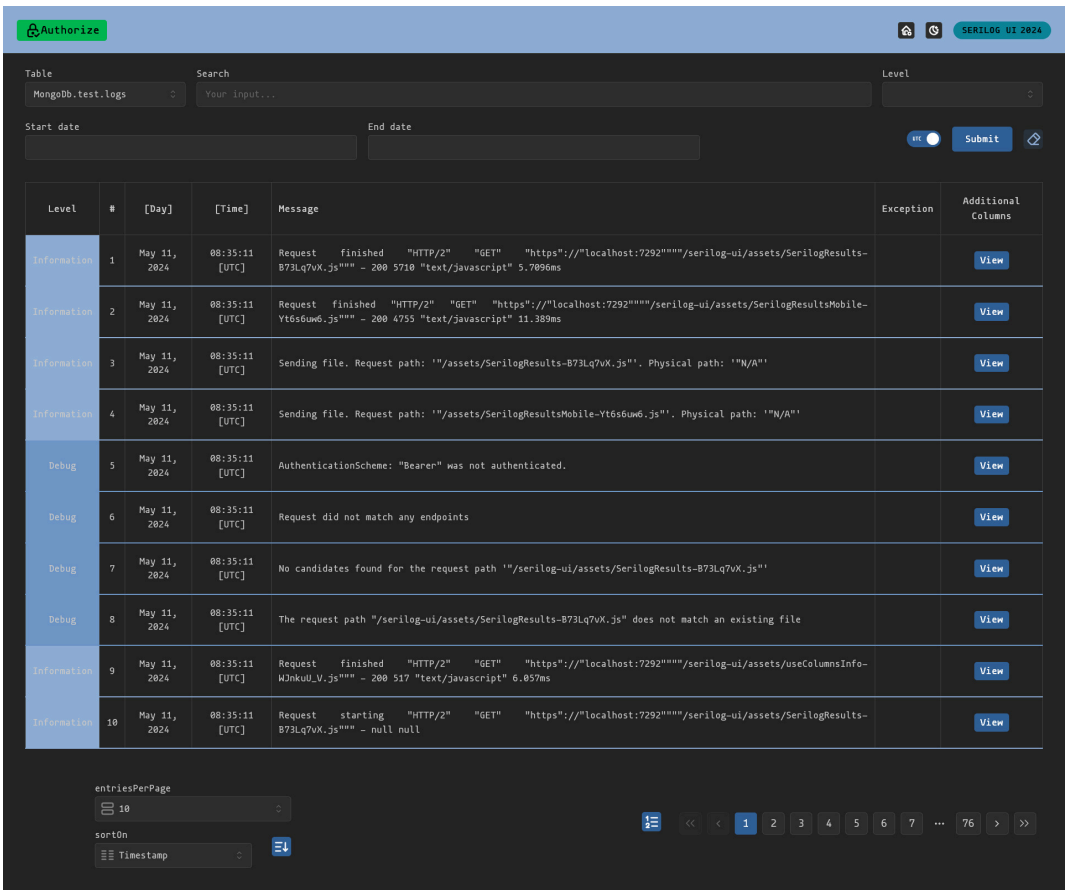

Imagen 2: Interfaz de serilog-ui mostrando logs almacenados en un sink MongoDB

El lenguaje principal es C#, al ser una extensión para el framework .NET, mientras que para el dashboard utiliza TypeScript/JavaScript y CSS. También se usa Powershell y shell, entre otros. El proyecto emplea un sistema híbrido de construcción automatizado mediante GitHub Actions: MSBuild para el backend (archivos .sln) y Yarn/npm para el frontend (package.json). Esta plataforma de CI gestiona flujos de trabajo (*workflows*) para ejecutar pruebas unitarias y validar el código antes de publicar en NuGet (el gestor de paquetes que permite distribuir librerías de .NET). Además, incluye scripts de automatización (.ps1 y .sh) que permiten replicar la construcción del software en cualquier entorno sin intervención manual.

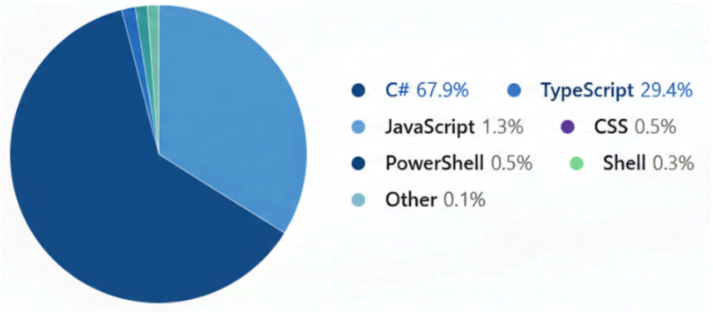

Imagen 3: Lenguajes del proyecto

Registrado oficialmente en la API de GitHub el 4 de abril de 2020, el proyecto se considera actualmente activo y bajo mantenimiento. Aunque el último release estable (v3.2.0) data de abril de 2025, la actividad técnica es constante en 2026, con Pull Requests y gestión de Issues en enero y febrero que garantizan su viabilidad operativa y seguridad.

El repositorio registra 65 issues (60 cerrados y 5 abiertos), lo que refleja una resolución de incidencias muy activa con una tasa de cierre superior al 90%. La mayoría usan etiquetas de tipo “bug” (errores), “enhancement” (mejoras), “help wanted” y “question”. Muchos de ellos se centran en la compatibilidad con bases de datos, un ejemplo es el Issue #108, donde se corrige el formato de columnas para PostgreSQL tras el reporte de los usuarios.

El proyecto tiene una popularidad consolidada con 434k descargas en NuGet, 262 estrellas, 8 seguidores y 44 forks. Su desarrollo es colaborativo, con 14 contribuidores liderados por Matteo Gregoricchio (followynne) y Mohsen Ismael Pour (mo-esmp). Se distribuye bajo la licencia MIT, lo que permite su integración en proyectos comerciales y privados con mínimas restricciones legales.

Declaración del uso de IA		
Estudiante (s)	**Enlace a la conversación de chat (Chat GPT, Gemini, Copilot, etc.)**	**Modelo (indicar proveedor y versión)**
Javier Martín	https://chatgpt.com/share/6991c33a-e3c8-800a-898b-fbf3cc3084c7	GPT-5

2 Visión general

Para ilustrar el uso y la integración de serilog-ui, nos basamos en algunos de los ejemplos de implementación contenidos en el propio repositorio del proyecto.

A. Configuración de Servicios y Proveedores (Registro)

En el archivo samples/WebApi/Extensions/ServiceCollectionExtensions.cs, se observa cómo se registran los servicios de la interfaz y se configuran múltiples proveedores de datos (en este caso, SQL Server y un ejemplo condicional de ElasticSearch):

```
internal static IServiceCollection AddSerilogUiSample(this IServiceCollection services, IConfiguration configuration, bool enableElasticSample)
    => services
        .AddSerilogUi(options =>
            {
                options
                    /* Muestra de filtro de autorización por defecto */
                    .AddScopedAuthorizeLocalRequestsAuthFilter()

                    /* Registro de múltiples proveedores SQL Server */
                    .UseSqlServer<TestLogModel>(opt => opt
                        .WithConnectionString(configuration.GetConnectionString("MsSqlDefaultConnection")!)
                        .WithTable("logs"))
                    .UseSqlServer(opt => opt
                        .WithConnectionString(configuration.GetConnectionString("MsSqlBackupConnection")!)
                        .WithTable("logsBackup"))
                    .UseElasticSample(enableElasticSample);
            }
        );
```

El uso de múltiples contenedores de datos (como SQL Server y ElasticSearch simultáneamente) demuestra que el sistema implementa un **patrón de repositorio abstracto**. Esto permite que la interfaz unifique la visualización sin importar que el origen de los datos sea relacional (SQL) o basado en documentos (NoSQL).

B. Activación del Middleware en el Pipeline

En el archivo samples/WebApi/Program.cs, se identifica la activación de la interfaz visual y la personalización de su acceso (cambiando el prefijo de la ruta y ocultando la marca por defecto):

```
app.UseSerilogUi(options => options
    .WithRoutePrefix("my-test") // Acceso personalizado en /my-test
    .HideSerilogUiBrand());     // Personalización estética
```

Aquí se evidencia el uso de la aplicación como un **middleware no intrusivo** que se acopla al pipeline de ejecución de ASP.NET Core.

La capacidad de ocultar la marca propia (HideSerilogUiBrand) y personalizar la ruta de acceso no es solo estética, facilita que la herramienta se integre en productos comerciales como si fuera una funcionalidad nativa del propio software y no una librería externa.

C. Configuración Avanzada y Autenticación

Finalmente, en samples/WebApp/Program.cs, se muestra un uso más complejo que incluye seguridad mediante tokens JWT e inyección de scripts personalizados:

```
app.UseSerilogUi(options => options
    .WithHomeUrl("/#Test")
    .WithAuthenticationType(AuthenticationType.Jwt) // Uso de seguridad JWT
    .WithExpandedDropdownsByDefault()
    .EnableAuthorizationOnAppRoutes()
    .InjectJavascript($"/{serverSubPath}js/serilog-ui/custom.js") // Inyección de lógica JS propia
);
```

Este ejemplo ilustra cómo la herramienta permite integrarse en entornos que requieren control de acceso y personalización extendida.

La inyección de archivos externos (InjectJavascript e InjectStylesheet) convierte a serilog-ui en una plataforma altamente maleable. Permite adaptar la experiencia de usuario a los estándares de accesibilidad o identidad visual de cualquier corporación sin necesidad de recompilar la librería original.

Declaración del uso de IA		
Estudiante (s)	**Enlace a la conversación de chat (ChatGPT, Gemini, Copilot, etc.)**	**Modelo (indicar proveedor y versión)**
Pablo García	https://drive.google.com/file/d/1_bMNh2ImbL7AGEM0fOnDTBH5UCMJGAMk/view?usp=sharing *	Gemini 3 Fast

3 Participantes

Para el análisis de los interesados en el ecosistema de serilog-ui, se aplica la taxonomía de Wood, clasificando a los agentes según su responsabilidad y nivel de interacción con el sistema:

- **Desarrolladores (Developers):** Representan el núcleo técnico responsable del ciclo de vida del software. Su labor trasciende la mera escritura de código, ya que deben configurar la lógica de captura de eventos, la gestión de *sinks* y la arquitectura de la interfaz gráfica. En este proyecto, los desarrolladores actúan también como testeadores, implementando suites de pruebas unitarias para mitigar riesgos, y como comunicadores, al ser responsables de mantener la documentación técnica y el archivo README que define la visión estratégica y los objetivos del proyecto para la comunidad de código abierto.
- **Soporte (Support):** Personal encargado de la gestión de incidencias y el mantenimiento correctivo. Este rol se centraliza en la monitorización de la sección "Issues" de GitHub, donde se gestionan los errores reportados por la comunidad. Los participantes Dion Leurink y Mohsen Esmailpour encabezan este rol, cuya actividad principal se basa en la detección reactiva de fallos y la provisión de soluciones técnicas que aseguren la continuidad operativa de la librería en diferentes entornos.

- **Usuarios (Users):** Consumidores finales de la herramienta, generalmente integrados por desarrolladores de software o administradores de sistemas que utilizan el *dashboard* de serilog-ui para la monitorización de APIs. A diferencia de los desarrolladores del núcleo, su interacción con el sistema se limita al consumo de información; su capacidad de acción se restringe a la consulta, filtrado y análisis de los logs registrados para el diagnóstico de errores en tiempo real, sin intervención en la configuración del flujo de datos.
- **Evaluadores (Assessors):** Agentes encargados de supervisar la conformidad del sistema con los estándares de calidad y seguridad preestablecidos. En este escenario, el rol de evaluador es desempeñado de forma automatizada por la plataforma SonarCloud. Este sistema actúa como un auditor externo que supervisa cada *Pull Request*, analizando métricas críticas como la deuda técnica, la cobertura de pruebas y posibles vulnerabilidades de seguridad, garantizando así que el código integrado cumpla con los umbrales de fiabilidad exigidos para una distribución estable.

Declaración del uso de IA		
Estudiante (s)	**Enlace a la conversación de chat (ChatGPT, Gemini, Copilot, etc.)**	**Modelo (indicar proveedor y versión)**
Alejandro Marquez	https://gemini.google.com/share/755a6cf082e6	Gemini 3
Gonzalo de la Fuente	https://gemini.google.com/share/e719639c4349	Gemini 3.1 Pro
Javier Martín	https://gemini.google.com/app/18c1a658b520b4ea?hl=es-ES	Gemini 3 fast

4 Vistas

El diseño arquitectónico de serilog-ui se puede describir utilizando las siguientes vistas.

4.1 Vista de contexto

La presente vista describe la relación y las interdependencias entre el sistema y su contexto externo, incluyendo cómo se relaciona con otros sistemas, entidades o individuos.

El proyecto se aloja en GitHub, donde se gestiona el control de versiones, el seguimiento de incidencias y la documentación (incluyendo una wiki y un readme con guías de instalación, configuración y desarrollo). El repositorio está integrado con herramientas de soporte en la nube como GitHub Actions para flujos de integración y despliegue continuos, SonarCloud para el análisis de calidad de código y métricas, y NuGet para la distribución y gestión de paquetes.

Los principales desarrolladores son colaboradores de la comunidad, con figuras clave como Mohsen Esmailpour y Matteo Gregoricchio liderando las contribuciones de código, revisiones y la gestión del proyecto, junto con más de 12 colaboradores adicionales que han participado activamente.

Serilog.ui es utilizado principalmente por desarrolladores .NET y equipos DevOps que trabajan con logging basado en Serilog en sus aplicaciones. Permite visualizar y consultar logs procedentes de diversos sinks de manera sencilla y centralizada, sin necesidad de acceder directamente a las bases de datos o archivos subyacentes.

Como se muestra en el diagrama, Serilog.UI se desarrolla mediante una combinación de tecnologías backend y frontend: C# para la biblioteca principal y los proveedores .NET, TypeScript y JavaScript para la interfaz web interactiva, CSS para el estilo visual, MSBuild para la compilación del backend (a través del archivo de solución .sln), y npm/Yarn para la gestión de dependencias del frontend. Se emplean herramientas de scripting multiplataforma como PowerShell y Shell para la automatización de builds.

El proyecto depende fundamentalmente de Serilog como framework de logging base, junto con sinks específicos de Serilog (por ejemplo, para MSSQL, PostgreSQL, MongoDB, Elasticsearch, MySQL/MariaDB, RavenDB, SQLite, entre otros) que permiten la conexión y recuperación de datos de logs desde distintos almacenamientos.

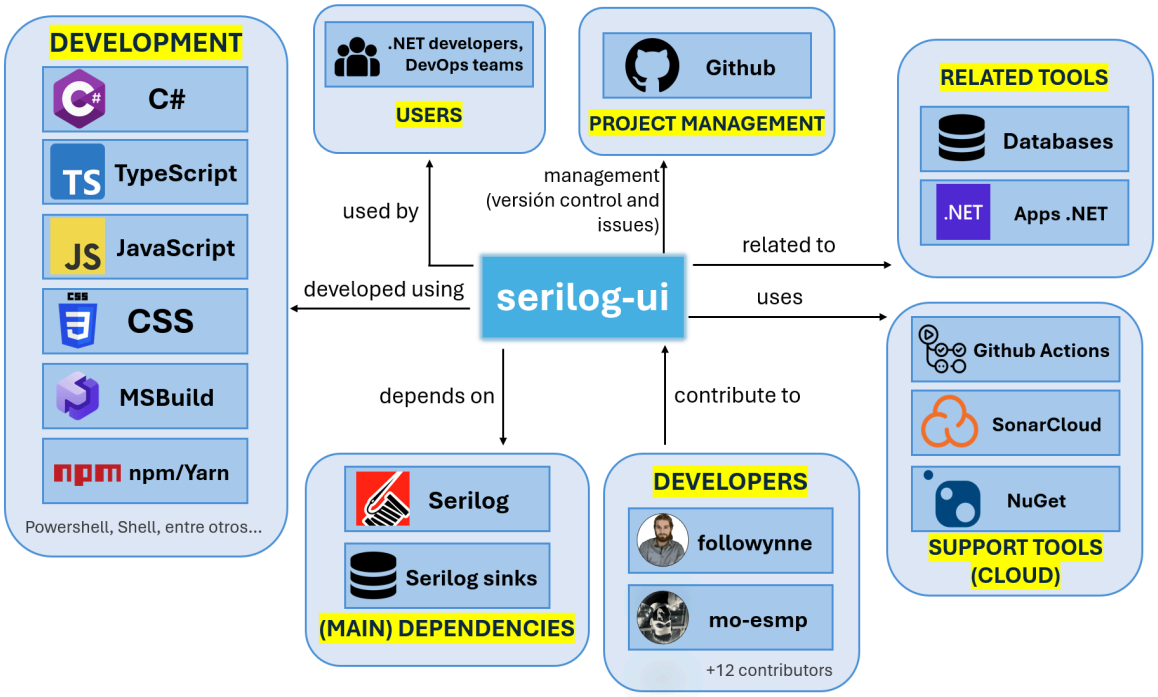

Imagen 4: Diagrama de contexto (realizado con Microsoft Visio)

Declaración del uso de IA		
Estudiante (s)	**Enlace a la conversación de chat (ChatGPT, Gemini, Copilot, etc.)**	**Modelo (indicar proveedor y versión)**
Javier Martín	https://chatgpt.com/share/6991c33a-e3c8-800a-898b-fbf3cc3084c7 https://grok.com/share/c2hhcmQtMi1jb3B5_bc10c664-2691-4eb4-abf0-fa7010f69bde	GPT -5 Gemini 3 Fast

4.2 Escenarios de uso

A continuación se describen los principales escenarios de uso identificados en el sistema serilog-ui, así como los actores que intervienen en cada uno de ellos.

A. Configurar serilog-ui
Actor: Desarrollador/DevOps
El desarrollador integra serilog-ui en una aplicación .NET mediante el uso de los métodos AddSerilogUi() y UseSerilogUi(). Esta configuración habilita los servicios necesarios y el middleware correspondiente para exponer la interfaz web de visualización de logs.

B. Generar logs
Actor: Aplicación .NET
La aplicación .NET, previamente configurada con la librería Serilog, genera eventos durante su ejecución que se almacenan en un sink configurado.

C. Acceder a la interfaz web
Actor: DevOps/User
El usuario accede desde el navegador a la ruta configurada de serilog-ui con el fin de visualizar los registros almacenados.

D. Consultar logs
Actor: Usuario
El sistema recupera los logs almacenados en el sink y los presenta en la interfaz gráfica, permitiendo su visualización estructurada.

E. Filtrar o buscar logs
Actor: Usuario
El usuario aplica filtros basados en criterios como fecha o contenido textual, con el objetivo de localizar eventos específicos dentro del conjunto de registros.

F. Analizar eventos
Actor: Usuario
El usuario inspecciona en detalle un evento concreto para realizar tareas de diagnóstico, monitorización o análisis del comportamiento de la aplicación.

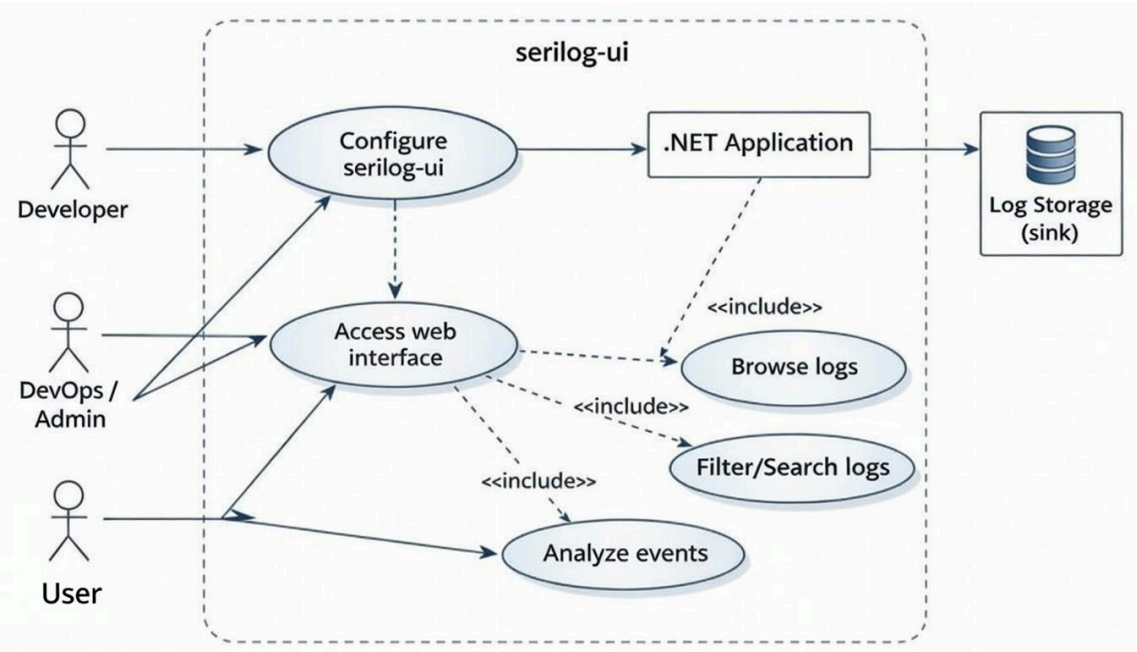

Imagen 5: Diagrama de casos de uso (Vista de escenarios)

Declaración del uso de IA		
Estudiante (s)	**Enlace a la conversación de chat (ChatGPT, Gemini, Copilot, etc.)**	**Modelo (indicar proveedor y versión)**
Alejandro Marquez	https://gemini.google.com/share/34ea3db77890	Gemini 3
Javier Martín	https://chatgpt.com/share/6991c33a-e3c8-800a-898b-fbf3cc3084c7	GPT-5

4.3 Vista funcional

La presente vista detalla los elementos funcionales del sistema serilog.ui, especificando sus responsabilidades, las interfaces que exponen y sus interacciones fundamentales.

El sistema ha sido concebido mediante una arquitectura basada en *plugins* o el patrón proveedor (*Provider Pattern*). Esta aproximación, que se distingue por una elevada modularidad, facilita el desacoplamiento total de la interfaz gráfica y la lógica de orquestación respecto a la tecnología de persistencia subyacente. A nivel de contexto, serilog.ui actúa como un *middleware* hospedado dentro de una aplicación ASP.NET Core (Host Application), cuya función es procesar las peticiones de un usuario a través del navegador y extraer, en modo lectura, los registros almacenados en sistemas de bases de datos externos.

Para lograr este objetivo, el sistema se articula internamente como un ensamblaje de componentes lógicos autónomos (distribuidos a través de paquetes NuGet) que interactúan entre sí de la siguiente manera:

Nivel de Componentes

El sistema se desglosa a continuación para revelar su arquitectura interna. Se especifican los submódulos lógicos que lo componen, así como las interacciones entre ellos. Este nivel de detalle ilustra cómo el sistema implementa un diseño modular basado en interfaces (patrón proveedor) para lograr un alto grado de desacoplamiento tecnológico.

- **Frontend (Capa de Presentación):** Aplicación de página única (SPA) desarrollada con React. Su cometido es capturar las acciones del usuario (filtros, búsquedas) en el navegador y renderizar la tabla de resultados mediante invocaciones API.
- **Core Engine (Middleware & API):** Es el componente orquestador del sistema. Su responsabilidad recae en interceptar las peticiones HTTP, servir los archivos web (HTML/JS) al usuario y exponer una API REST interna que actúa como interfaz entre el Frontend y la capa de datos.
- **Data Providers (Proveedores de Datos):** Desempeñan la función de "traductores" del sistema. Son módulos especializados e intercambiables encargados de tomar la petición genérica del Core y transformarla en una consulta nativa comprensible por la base de datos configurada (ej. sentencias SQL o consultas de Mongo).
- **Authorization Filters (Seguridad):** Componente de vigilancia que se ejecuta previamente a la entrega de cualquier dato. Evalúa las reglas de acceso establecidas por el programador (por ejemplo, verificar la autenticación del usuario o la validez de su dirección IP) para la protección de la información sensible.
- **Core Library (Dominio y Contratos):** Constituye la estructura estática fundamental del proyecto. Contiene los modelos de datos compartidos (como la estructura estándar de un log) y las interfaces (como IDataProvider). Facilita la comunicación estandarizada y con bajo acoplamiento entre todos los componentes.

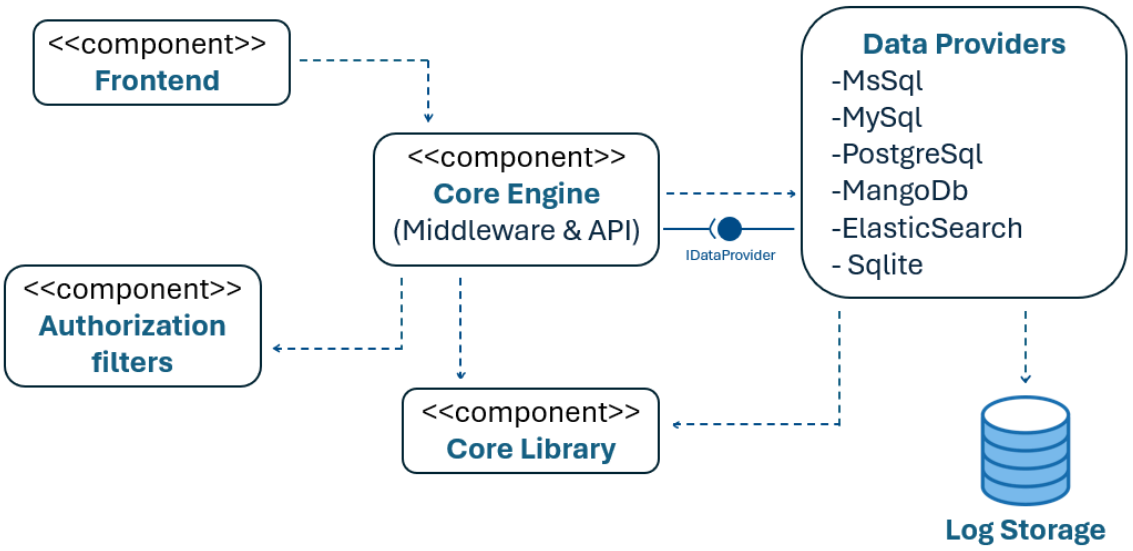

Imagen 6: Diagrama de componentes

Declaración del uso de IA		
Estudiante (s)	**Enlace a la conversación de chat (ChatGPT, Gemini, Copilot, etc.)**	**Modelo (indicar proveedor y versión)**
Gonzalo de la Fuente	https://gemini.google.com/share/e719639c4349 https://chatgpt.com/share/69a02afc-92c8-8004-ba62-92752820e01c	Gemini 3.1 Pro Chat GPT 5
Angel Nolasco	https://gemini.google.com/share/dadf0eaa5f6d	Gemini 3.1
Javier Martín	https://chatgpt.com/share/6991c33a-e3c8-800a-898b-fbf3cc3084c7	GPT-5

4.4 Vista de despliegue

Dado que serilog-ui se distribuye como una librería de clases (paquete NuGet), no se despliega como una aplicación independiente, sino que se incrusta como un artefacto dentro de una aplicación web anfitriona (Host Application). Para ilustrar este escenario, hemos identificado tres nodos principales en el diagrama UML de despliegue:

A. Dispositivo del Usuario (User Device):

Representa el hardware (PC, smartphone, etc.) utilizado por el usuario final (ej. un administrador de sistemas o desarrollador).

- Navegador Web (Browser): Actúa como el entorno de ejecución en el lado del cliente.
- Artefacto: En él se cargan e interpretan los archivos estáticos del Frontend de serilog-ui (compuestos por HTML, JavaScript compilado desde TypeScript y CSS).

B. Servidor de Aplicaciones (Application Server):

Es la máquina física, máquina virtual o contenedor (ej. Docker) donde se aloja el backend.

- Entorno de ejecución .NET: El motor en tiempo de ejecución (ASP.NET Core / Kestrel o IIS) que soporta la aplicación.
- Artefactos: Dentro de este entorno se despliega la Aplicación Anfitriona (.NET App). En su interior, reside el artefacto serilog-ui.dll (y los .dll de los correspondientes Data Providers instalados vía NuGet).

C. Servidor de Base de Datos (Database Server):

Representa la infraestructura donde persisten físicamente los logs.

- Entorno de ejecución DBMS: El sistema gestor de bases de datos elegido (ej. Microsoft SQL Server, MongoDB, Elasticsearch).
- Artefacto: La base de datos de logs propiamente dicha.

La comunicación entre el Dispositivo del Usuario y el Servidor de Aplicaciones se realiza mediante el protocolo HTTP/HTTPS, sirviendo tanto las vistas web como atendiendo las llamadas a la API interna del dashboard. Por su parte, la comunicación entre el Servidor de Aplicaciones (a través de los Data Providers de serilog-ui) y el Servidor de Base de Datos se realiza a través de los puertos y protocolos nativos TCP/IP del motor de base de datos correspondiente.

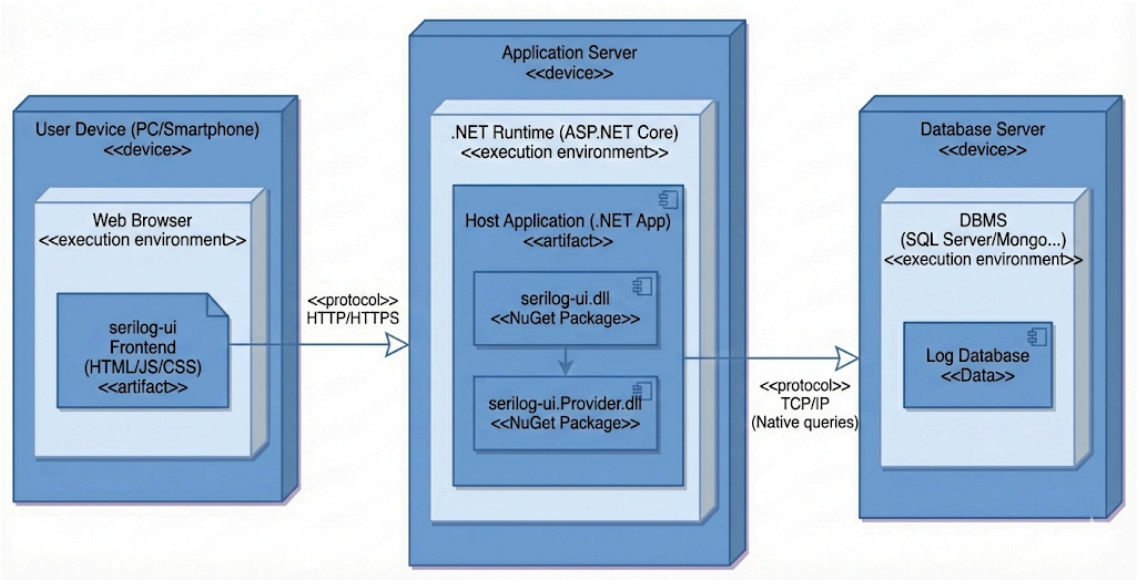

Imagen 7: Diagrama de despliegue

Declaración del uso de IA		
Estudiante (s)	**Enlace a la conversación de chat (ChatGPT, Gemini, Copilot, etc.)**	**Modelo (indicar proveedor y versión)**
Gonzalo de la Fuente	https://gemini.google.com/share/e719639c4349	Gemini 3.1 Pro

4.5. Vista de Desarrollo

La Figura muestra un diagrama de paquetes UML de serilog-ui generado automáticamente con la herramienta UML Lab. El proyecto se distribuye a lo largo de 64 paquetes, 94 archivos, 92 clases, 387 métodos y 2665 líneas de código. Contiene un total de 951 líneas comentadas, lo que representa el 26,4% respecto al total.

Destacan los métodos AddSerilogUi() y UseSerilogUi(), que constituyen los puntos de integración del proyecto en una aplicación ASP.NET Core. El primero registra los servicios necesarios para acceder a los logs almacenados en los sinks configurados, mientras que el segundo incorpora el middleware responsable de exponer la interfaz web y atender las peticiones del navegador. En conjunto, ambos permiten conectar el almacenamiento de registros con su visualización a través de la UI.

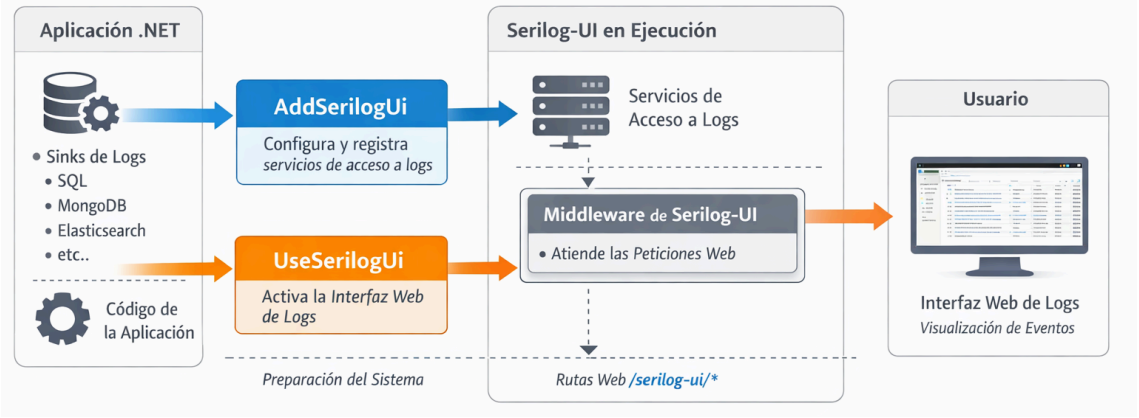

Imagen 8: Esquema funcionamiento

En la siguiente tabla podemos ver de forma detallada la distribución de las clases y líneas de código entre los principales paquetes en los que se organiza el código. Podemos comprobar que tienen un peso más relevante los paquetes Core y Web.

Paquete	Líneas de código	Clases
Serilog.Ui.Core	474	22
Serilog.Ui.ElasticSearchProvider	183	5
Serilog.Ui.MongoDbProvider	208	4
Serilog.Ui.MsSqlServerProvider	248	8
Serilog.Ui.MySqlProvider	259	11
Serilog.Ui.PostgreSqlProvider	277	9
Serilog.Ui.RavenDbProvider	159	4
Serilog.Ui.SqliteDataProvider	167	5
Serilog.Ui.Web	690	24
Total	**2665**	**92**

El siguiente diagrama detalla la estructura y jerarquía de paquetes del código fuente (src) del proyecto.

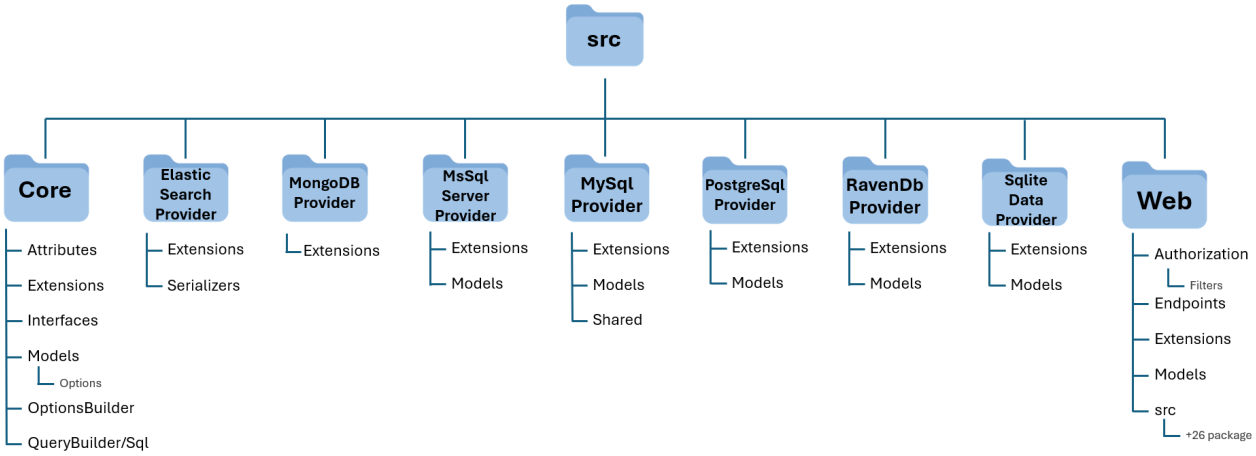

Imagen 9: Diagrama de paquetes

Desde el punto de vista estructural podemos diferenciar tres tipos de paquetes según su responsabilidad. Serilog.Ui.Core constituye el núcleo del sistema y define la lógica común, los modelos y las interfaces para el acceso a los logs. Serilog.Ui.Web implementa la capa de presentación, incluyendo el middleware y los recursos que permiten exponer la interfaz web de visualización. Finalmente, los paquetes *Provider* (MsSqlServer, MySql, PostgreSql, MongoDb, ElasticSearch, RavenDb y Sqlite) actúan como adaptadores para distintos sistemas de almacenamiento, implementando las interfaces del núcleo para recuperar los logs desde cada tecnología específica.

El proyecto se sustenta en dos ecosistemas principales que dependen de las siguientes librerías:

A. Core y Backend (.NET)

- **Microsoft.AspNetCore.App:** Framework base para integrar la interfaz como middleware web.
- **Serilog:** Motor de logging fundamental del cual el proyecto extrae y estructura los datos.
- **Serilog.Ui.Core:** Biblioteca interna que abstrae la lógica de filtrado y acceso a los registros.
- **Database Sinks (Npgsql, MongoDB.Driver, etc.):** Adaptadores que permiten la lectura de logs en diversos motores de base de datos.
- **System.Text.Json:** Encargada de la serialización eficiente de datos entre el servidor y la interfaz.

B. Interfaz de Usuario (React/TypeScript)

- **React & TypeScript:** Tecnologías base para la construcción de una interfaz reactiva y con tipado seguro.
- **Mantine Suite (@mantine/core, dates, hooks):** Framework de diseño que proporciona todos los componentes visuales y la experiencia de usuario.
- **TanStack React Query:** Gestor de estado que optimiza las peticiones de logs, la caché y los tiempos de carga.
- **Dayjs:** Utilidad para el formateo de fechas y marcas temporales en un formato legible.
- **Tabler Icons (@tabler/icons-react):** Set de iconografía técnica para identificar niveles de log y acciones.
- **Jose:** Implementación de seguridad para el manejo de autenticación y tokens JWT.
- **Xml-formatter:** Herramienta para la indentación y visualización estructurada de logs en formato XML.

Declaración del uso de IA		
Estudiante (s)	**Enlace a la conversación de chat (ChatGPT, Gemini, Copilot, etc.)**	**Modelo (indicar proveedor y versión)**
Javier Martín	https://gemini.google.com/app/18c1a658b520b4ea?hl=es-ES	

5 Puntos de variabilidad y extensión

Los puntos de variabilidad son aquellas partes del sistema que pueden configurarse o sustituirse por diferentes alternativas sin modificar el núcleo de la aplicación. En el caso de *Serilog-ui*, se han identificado los siguientes puntos de variabilidad:

A. Proveedor de datos

El usuario puede elegir entre distintos sistemas de almacenamiento para la lectura de logs, instalando los paquetes NuGet indicados en el README (MSSqlServer, MySql, MariaDB, PostgreSql, MongoDB, ElasticSearch, RavenDB y SQLite). Esta capacidad de selección constituye un claro punto de variabilidad, ya que el sistema permite adaptarse a múltiples tecnologías de persistencia. Las bases de datos como MSSqlServer, MySql, MariaDB, PostgreSql y SQLite son relacionales, mientras que MongoDB, ElasticSearch y RavenDB siguen modelos documentales o orientados a búsqueda.

B. Ruta del Dashboard (Pipeline de Middleware)

El sistema permite configurar la ruta en la que se expone la interfaz web, así como su posición dentro del pipeline de middleware de ASP.NET Core. En el caso habitual de Serilog-ui, el middleware app.UseSerilogUi() se sitúa después de los middlewares de

autenticación (app.UseAuthentication) y autorización (app.UseAuthorization), permitiendo controlar el acceso a la visualización de logs. Esta flexibilidad en la configuración del pipeline constituye otro punto de variabilidad.

C. Herramientas de instalación y despliegue

El sistema puede instalarse y ejecutarse mediante diferentes herramientas del ecosistema .NET, como Visual Studio o la interfaz de línea de comandos dot.net. Aunque no afecta directamente a la lógica interna, esta posibilidad de elección en el entorno de desarrollo y despliegue representa un punto de variabilidad desde el punto de vista operativo.

Los **puntos de extensión** son aquellas partes del sistema diseñadas para facilitar la incorporación de nuevas funcionalidades en el futuro. En muchos casos coinciden con puntos de variabilidad, como sucede en Serilog-ui. Se identifican los siguientes puntos de extensión:

A. Nuevos proveedores de almacenamiento

La arquitectura basada en interfaces y proveedores independientes permite añadir fácilmente nuevos paquetes para soportar otras tecnologías de almacenamiento. El núcleo del sistema está preparado para integrar nuevas implementaciones sin modificar la lógica principal.

B. Interfaz de usuario

Al estar expuesta mediante middleware y recursos web (HTTP, JavaScript y CSS), la interfaz puede evolucionar o personalizarse. La separación entre núcleo y capa web facilita futuras mejoras visuales o cambios en la experiencia de usuario.

C. Funcionalidades y filtros

El sistema está preparado para incorporar nuevas funcionalidades relacionadas con la consulta y análisis de logs, como filtros adicionales, nuevos criterios de búsqueda o herramientas de diagnóstico. La estructura modular facilita la ampliación progresiva de estas capacidades.

Declaración del uso de IA		
Estudiante (s)	**Enlace a la conversación de chat (ChatGPT, Gemini, Copilot, etc.)**	**Modelo (indicar proveedor y versión)**
Alejandro Márquez	https://gemini.google.com/share/413a599177c5	Gemini 3

6 Análisis de atributos de calidad

Mantenibilidad: se refiere a la facilidad para modificar el sistema, introducir mejoras o corregir errores. La mantenibilidad de serilog-ui es buena gracias a su arquitectura modular, que separa la lógica del servidor en .NET de la interfaz de usuario. El proyecto utiliza tipado fuerte con TypeScript para asegurar la consistencia del código, lo que coincide con la calificación de "A" en SonarCloud. No obstante, su calificación de mantenibilidad no es superior debido a la carencia de una wiki detallada o documentación técnica extensa, lo que supone una barrera para nuevos desarrolladores a pesar de tener una deuda técnica baja.

Seguridad: se refiere a la capacidad del sistema para evitar el acceso no autorizado a los registros de eventos. Serilog-ui funciona como una interfaz de visualización integrada en ASP.NET Core, por lo que su seguridad depende del host, aunque permite configurar políticas de autorización y gestión de tokens (JWT) mediante la librería jose. Al ser una herramienta de consulta y no de ejecución, su superficie de ataque es reducida, manteniendo una calificación de "A" en SonarCloud, lo que garantiza un código libre de vulnerabilidades críticas.

Reutilización: facilidad con la que partes del sistema pueden emplearse en otros proyectos. Es aceptable debido a su diseño modular, que aísla la lógica de acceso a datos en el proyecto Serilog.Ui.Core. Esto permite que los componentes de lectura de registros sean integrables en otras herramientas de monitorización, aunque la interfaz de usuario está estrechamente acoplada a la API interna del proyecto.

Rendimiento: mide la eficiencia en el uso de tiempo y memoria. En serilog-ui es alto porque usa "paginación", cargando solo unos pocos logs a la vez para no bloquear el navegador. Esto permite que las búsquedas sean instantáneas y fluidas sin consumir apenas recursos, incluso con grandes volúmenes de datos.

Usabilidad: mide la facilidad de uso del sistema. Es buena, ya que su instalación es directa mediante un paquete NuGet y ofrece un panel de visualización intuitivo para filtrar registros sin configuración previa. Sin embargo, se carece de documentación técnica actualizada para desarrolladores que deseen extender o personalizar las funcionalidades del dashboard.

Declaración del uso de IA		
Estudiante (s)	**Enlace a la conversación de chat (ChatGPT, Gemini, Copilot, etc.)**	**Modelo (indicar proveedor y versión)**
Javier Martín	https://gemini.google.com/app/18c1a658b520b4ea?hl=es-ES	Gemini 3 fast

7 Sugerencias de mejora

Basándonos en los análisis de SonarCloud podemos decir que el proyecto tiene una excelente cobertura y una evaluación muy positiva de sus atributos de calidad. No obstante, en base a nuestro análisis del proyecto, así como a partir de las recomendaciones de los Community Standards de Github hemos detectado varios aspectos en los que serilog-ui podría considerar mejoras.

A. Documentación Técnica.

Actualmente, el proyecto carece de una guía de arquitectura y extensibilidad, lo que dificulta que nuevos desarrolladores entiendan el funcionamiento interno o creen nuevos conectores de bases de datos sin leer todo el código fuente. Asimismo, la ausencia de una Wiki detallada y de documentación sobre su API interna limita la capacidad de personalizar la interfaz o integrar los datos en otras herramientas. Centralizar manuales de configuración avanzada y guías de contribución reduciría la barrera de entrada, facilitando que la comunidad pueda mantener y evolucionar el sistema.

B. Código de conducta.

Se ha detectado la ausencia de un código de conducta por parte de GitHub, el cual es fundamental para establecer normas claras para los colaboradores, siendo esencial para el mantenimiento de un proyecto de código abierto por parte de desarrolladores externos.

C. Política de Seguridad.

El proyecto carece de una política de seguridad formal, lo que supone un riesgo para la divulgación responsable de vulnerabilidades.
Una mejora crítica sería establecer un protocolo oficial que defina cómo reportar fallos de seguridad de forma privada, qué versiones del software cuentan con soporte activo y los tiempos de respuesta esperados. Esto aumentaría la confianza de los usuarios corporativos y garantizaría que los parches de seguridad se gestionen antes de que los fallos se hagan públicos.

D. “Pull Request” (estandarización de contribuciones)

El repositorio carece de plantillas para Pull Requests, lo que provoca que las propuestas de mejora lleguen sin un formato unificado. Implementar una plantilla estandarizada obligaría a los colaboradores a documentar sus cambios, enlazar los problemas

resueltos y confirmar que se han pasado los tests de calidad. Esto agilizaría la revisión de código por parte de los mantenedores y aseguraría que ninguna contribución rompa funcionalidades existentes por falta de información.

E. Seguridad en el código.

Serilog-ui, un *middleware* de la aplicación principal, no dispone de limitación de peticiones a la base de datos de logs. Un uso excesivo (consultas complejas o clics reiterados) podría resultar en la saturación de la base de datos y el agotamiento de los recursos del servidor, lo que potencialmente causaría un ataque de Denegación de Servicio (DoS) y el fallo de la aplicación anfitriona.
Se propone implementar *Rate Limiting* (Limitación de Tasa) en las rutas de serilog-ui utilizando el *middleware* nativo de ASP.NET Core (Microsoft.AspNetCore.RateLimiting). Esta medida permitiría restringir la cantidad de búsquedas (ej., a 30 por minuto por usuario/IP), protegiendo significativamente la Disponibilidad del sistema sin afectar la experiencia del usuario habitual.

F. Dimensión de Pruebas y Calidad

A pesar de la existencia de pruebas unitarias para el *backend* y el *frontend*, la validación integral de la interfaz gráfica y los proveedores de bases de datos no se encuentra automatizada. Se aconseja la integración de pruebas funcionales automatizadas End-to-End (E2E). Esto posibilitaría la simulación de la experiencia real del usuario en los flujos de Integración Continua (CI), previniendo regresiones visuales.

Declaración del uso de IA		
Estudiante (s)	**Enlace a la conversación de chat (ChatGPT, Gemini, Copilot, etc.)**	**Modelo (indicar proveedor y versión)**
Alejandro Márquez	https://gemini.google.com/share/1809fb834ccc	Gemini 3
Gonzalo de la Fuente	https://gemini.google.com/share/e719639c4349	Gemini 3.1 Pro
Javier Martín	https://gemini.google.com/app/18c1a658b520b4ea?hl=es-ES	Gemini 3 fast

8 Contribuciones al proyecto

Nuestra contribución al proyecto serilog-ui se ha centrado en la refactorización de los componentes de acceso a datos para reducir significativamente la deuda técnica (code

smells de código duplicado) y mejorar los atributos de calidad de mantenibilidad y cohesión.

Enlace al commit en nuestro fork creado para poder realizar commit con las modificaciones sin modificar el repositorio original:

[https://github.com/PGT5772/serilog-ui/commit/21fb7aa09cae7b878986ab46b5f32a756063a7a6]

Descripción detallada de la contribución:

Tras analizar la arquitectura de providers (proveedores de bases de datos), detectamos que la lógica de construcción de consultas SQL para el filtrado de logs (cláusulas WHERE) estaba completamente duplicada en las clases SqlServerQueryBuilder, PostgresQueryBuilder, MySqlQueryBuilder y SqliteQueryBuilder.

Para solucionar esto sin romper la compatibilidad de los diferentes dialectos SQL, hemos aplicado el patrón de diseño estructural "Template Method":

Hemos extraído la lógica común (GenerateWhereClause) y la hemos movido al núcleo del sistema (clase base SqlQueryBuilder en Serilog.Ui.Core).

Hemos estandarizado la construcción condicional utilizando la cláusula genérica WHERE 1 = 1.

Hemos definido un método virtual EscapeColumn() (el gancho del Template Method) en el núcleo, que permite el polimorfismo.

Hemos eliminado cientos de líneas de código redundante en los proveedores y hemos especializado aquellos que requerían sintaxis propia sobrescribiendo el método gancho (usando [] para SQL Server y "" para PostgreSQL).

Esta modificación descentraliza la responsabilidad repetitiva, garantizando que futuras expansiones en los filtros de búsqueda sólo tengan que programarse una vez en el componente Core.

Declaración del uso de IA		
Estudiante (s)	**Enlace a la conversación de chat (ChatGPT, Gemini, Copilot, etc.)**	**Modelo (indicar proveedor y versión)**
Pablo García	https://drive.google.com/file/d/10iw-ATyroR_X8dfkC80deyk91MJ8OX-c/view?usp=sharing *	Gemini 3.1 Pro

9 Conclusiones

En este proyecto hemos documentado el diseño arquitectónico de serilog-ui, herramienta que ofrece una interfaz web para visualizar y filtrar logs de Serilog. Su

naturaleza como *middleware* permite una integración no intrusiva en sistemas .NET, facilitando la monitorización técnica en tiempo real sin alterar el código de negocio. Este diseño modular garantiza una gestión de eventos centralizada, eficiente y estéticamente adaptada a la aplicación anfitriona.

La arquitectura basada en proveedores de datos asegura una alta extensibilidad, aunque se han detectado oportunidades de mejora en la automatización de pruebas E2E y en la estandarización de procesos mediante *PR Templates*. Nuestra contribución se ha centrado en optimizar el filtrado en el núcleo (*Core*) mediante el patrón *Template Method*, reduciendo la redundancia de código. En definitiva, un diseño desacoplado y una documentación rigurosa son los factores determinantes para lograr una solución escalable y profesional.

Declaración del uso de IA		
Estudiante (s)	**Enlace a la conversación de chat (ChatGPT, Gemini, Copilot, etc.)**	**Modelo (indicar proveedor y versión)**
Alejandro Marquez	https://gemini.google.com/share/3d2671b13de2	Gemini 3
Gonzalo de la Fuente	https://gemini.google.com/share/e719639c4349	Gemini 3.1 Pro
Javier Martín	https://gemini.google.com/app/18c1a658b520b4ea?hl=es-ES	Gemini 3 fast

Referencias

* Las conversaciones marcadas están iniciadas desde un agente de Gemini (Gem) creado por nosotros y con las instrucciones básicas del proyecto así como los criterios de evaluación, la plantilla del proyecto y el proyecto de ejemplo. https://drive.google.com/file/d/10bYAsfRzWGx042sjP_lBq5F0pFzdlVTN/view?usp=sharing